

Fundamentación — GraphQL Gateway + microservicios REST

Proyecto: Plataforma de pedidos para una cadena de comidas rápidas **Documento:** fundamentación de la arquitectura de entrada **Relacionado con:** docs/requerimientos-backend-rest.md (variante REST) **Versión:** 1.0

Este documento justifica por qué la capa de entrada es un **GraphQL Gateway** y por qué los **microservicios internos** exponen **REST** (en lugar de GraphQL federado, gRPC o REST puro de punta a punta). Sirve de fundamentación de la decisión de arquitectura para la defensa del proyecto.

Índice

1. [Contexto](#)
 2. [El problema que resuelve el gateway](#)
 3. [Beneficios del GraphQL Gateway](#)
 4. [Por qué REST en los microservicios](#)
 5. [Alternativas consideradas y por qué se descartaron](#)
 6. [Decisión](#)
 7. [Consecuencias \(pros y contras\)](#)
 8. [Reglas de la frontera GraphQL/REST](#)
-

1. Contexto

La plataforma tiene **cinco frontends** (apps/auth, apps/store, apps/admin, apps/admin-global, apps/rider) y **tres servicios de dominio** (auth, commerce, delivery), cada uno dueño de sus colecciones en una base MongoDB única.

El problema de diseño es: **¿cómo exponen los servicios sus datos a los frontends?**

- Si cada frontend habla directo con los servicios, cada pantalla haría múltiples llamadas HTTP y armaría las uniones a mano (ej. un pedido necesita cliente, sucursal y productos → 3+ requests).
- Si exponemos **GraphQL por servicio**, cada servicio tendría que implementar su propio servidor GraphQL y seguiría sin resolver las uniones entre servicios.

- Si usamos **Apollo Federation**, la maquinaria de supergraph/subgraphs es desproporcionada para 3 servicios y agrega complejidad operativa sin beneficio visible.

La solución elegida separa dos preocupaciones distintas y aplica la mejor herramienta a cada una:

Hacia afuera (frontends): GraphQL. Un único contrato flexible, tipado y agregado. **Hacia adentro (servicios): REST.** Contratos simples, depurables y baratos de mantener.

2. El problema que resuelve el gateway

El gateway existe para **desacoplar el contrato externo (GraphQL) del contrato interno (REST)**. En una plataforma con frontends móviles (Tienda y Repartidor son mobile-first, Tienda además compila a Android con Capacitor), el costo de red importa: cada request extra y cada byte extra degradan la experiencia.

El GraphQL Gateway resuelve tres problemas concretos:

1. **N+1 entre servicios:** un pedido referencia cliente (`Auth`) y sucursal (`Commerce`). Sin gateway, el frontend haría 3 llamadas. Con el gateway, hace **una** query y el gateway resuelve las referencias internamente (con DataLoader para agrupar).
2. **Sobre-fetching / under-fetching:** cada pantalla pide solo los campos que muestra. La lista de pedidos no trae el detalle de cada ítem; el detalle sí. Esto es especialmente valioso en mobile.
3. **Contrato único y evolucionable:** los 5 frontends comparten un solo esquema tipado y versionable, en lugar de coordinar 5 integraciones REST distintas.

3. Beneficios del GraphQL Gateway

Beneficio	Descripción	Impacto en el proyecto
Un solo endpoint	Los 5 frontends consumen <code>POST /graphql</code> . No hay que coordinar decenas de URLs.	Menos configuración en <code>@repo/api</code> .
Consulta exacta por pantalla	Cada pantalla declara los campos que necesita; evita traer datos de más.	Menor payload en Tienda (Android) y Repartidor.

Beneficio	Descripción	Impacto en el proyecto
Agregación cross-service	El gateway resuelve <code>Order.client</code> , <code>Order.branch</code> , <code>Product.category</code> , etc. en una sola respuesta.	El frontend no conoce la topología interna de servicios.
Esquema tipado y compatible	El SDL se genera/exporta y se codegene en TypeScript para <code>@repo/domain</code> .	Tipos y validaciones consistentes con el backend.
Evolución sin romper clientes	Agregar campos es aditivo; los campos obsoletos se marcan <code>@deprecated</code> .	Se puede iterar el backend sin tocar todos los frontends a la vez.
Autenticación y RBAC centralizados	JWT, roles y rate limiting se aplican en la entrada.	Los servicios se mantienen más simples y protegidos en profundidad.
Herramientas de desarrollo	Sandbox, introspección y documentación del esquema.	Facilita el desarrollo y la defensa del proyecto.
DataLoader	Agrupar y deduplica las llamadas REST internas.	Elimina el problema N+1 entre servicios.

Ejemplo concreto: la pantalla T-13 (Seguimiento de pedido) necesita estado, sucursal, ítems, dirección e historial. Con REST directo serían ~4 endpoints; con GraphQL es una sola query y el gateway arma la respuesta.

4. Por qué REST en los microservicios

Los beneficios de GraphQL (consultas flexibles, selección de campos, evitar over-fetching) **no aplican al tráfico interno gateway→servicio**:

- El gateway **siempre** quiere el recurso completo (un `Order`, un `User`, un `Branch`), no un subconjunto de campos. La flexibilidad de GraphQL no aporta nada entre el gateway y el servicio.
- No hay un "usuario humano" ni una UI del otro lado de la llamada interna; no hay necesidad de un lenguaje de consulta.
- GraphQL por servicio reintroduciría el problema que intentamos evitar (cada servicio con su parser/validador y sin resolver las uniones).

Por eso, en la frontera interna se usa **REST**, que aporta:

Beneficio de REST interno	Descripción
Simpleza	Cada operación es un endpoint HTTP con método y ruta clara.
Depurabilidad	Cualquier llamada se reproduce con <code>curl</code> /Postman; los logs son legibles.
Contrato explícito (OpenAPI)	Un documento por servicio define rutas, schemas y errores; genera clientes tipados.
Encaje natural con NestJS	Los controllers de NestJS ya son REST; se alinea con la arquitectura por dominio.
Costo bajo de aprendizaje	El equipo (con trainees) ya conoce HTTP/JSON; no requiere dominio de protobuf ni GraphQL server-side.
Separación limpia	El servicio solo expone recursos; no sabe quién lo consume ni qué campos se piden.

Principio: GraphQL resuelve el problema del *consumidor* (frontend); REST resuelve el problema del *productor* (servicio). Al separarlos, cada capa usa la herramienta que resuelve su problema.

5. Alternativas consideradas y por qué se descartaron

Alternativa	Ventaja	Desventaja en este proyecto	¿Se eligió?
GraphQL Federation (Apollo)	Uniones entre subgraphs automáticas	Supergraph, router, <code>@key</code> , operativa extra para solo 3 servicios; curva alta	✗
GraphQL por servicio	Contrato tipado en cada servicio	Reintroduce complejidad sin resolver las uniones; duplica parser/validador	✗

Alternativa	Ventaja	Desventaja en este proyecto	¿Se eligió?
REST de punta a punta	Máxima simpleza	El frontend haría N+1 llamadas; payload sobredimensionado en mobile; contratos por frontend	✗ (solo como fallback)
gRPC interno	Contrato tipado fuerte + rendimiento	Requiere proto, codegen y tooling; mayor costo inicial para un equipo con trainees	⏏ (variante documentada, no elegida)
GraphQL Gateway + REST interno	Lo mejor de ambos mundos en cada frontera	Gateway agrega una capa (pero fina, sin lógica de negocio)	✓

Por qué **no** gRPC como opción principal, pese a ser técnicamente sólida: el beneficio real de gRPC (contrato binario tipado y baja latencia) no compensa, a esta escala y con este equipo, el costo de mantener `.proto`, generación de stubs y herramientas de debug. REST + OpenAPI logra un contrato tipado similar con mucha menos fricción.

6. Decisión

Se adopta la arquitectura Frontends → GraphQL Gateway → REST → microservicios.

- El **gateway** posee el esquema GraphQL, implementa los resolvers traduciendo a REST, y concentra JWT, RBAC, rate limiting, DataLoader y observabilidad. **No tiene lógica de negocio ni base de datos.**
- Los **servicios** exponen recursos REST versionados (`/v1`) documentados con OpenAPI, dueños de sus colecciones MongoDB. Siguen la arquitectura por dominio (`controller` → `orchestrator` → `servicio primario` → `repositorio`).
- La comunicación **asíncrona** entre servicios (eventos) se mantiene por broker (RabbitMQ/Kafka), igual que en la versión federada.

7. Consecuencias (pros y contras)

Ventajas

- Una sola query por pantalla para el frontend, con payload mínimo (clave en mobile).
- Servicios simples, depurables y alineados con NestJS.

- Frontera interna/externa claramente separada y con contratos versionados (OpenAPI + SDL).
- Menor costo de onboarding para el equipo.

Costos / riesgos a gestionar

- El gateway es una **capa adicional**; debe mantenerse fina (solo traducción) para no convertirse en un cuello de botella.
- El N+1 se traslada del frontend al gateway: se mitiga con **DataLoader** y, si hace falta, endpoints batch (`GET /v1/users?ids=...`).
- Dos contratos que mantener sincronizados (SDL del gateway y OpenAPI de servicios); se mitiga con codegen y tests de integración.
- Sin WebSocket/streaming por defecto: para el seguimiento en tiempo real se usa polling moderado (ya previsto en los requisitos funcionales).

8. Reglas de la frontera GraphQL/REST

Para que la decisión se mantenga coherente:

1. El gateway **no** implementa lógica de negocio; solo valida, traduce y compone.
2. Un campo GraphQL puede implicar **varias** llamadas REST, pero cada llamada REST corresponde a **un** recurso/operación del servicio.
3. Las uniones entre servicios (`Order.client` , `Order.branch` , etc.) se resuelven **siempre** en el gateway vía DataLoader; los servicios primarios nunca se llaman entre sí.
4. El contexto de identidad (`X-User-Id` , `X-Roles` , `X-Branch-Id`) viaja en headers y los servicios lo revalidan.
5. Todo cambio de contrato se versiona: `@deprecated` en GraphQL y `/v1` → `/v2` en REST.